សប្តាហ៍ទី 4 · Functions ដែលអាចទុកចិត្តបាន · Midterm

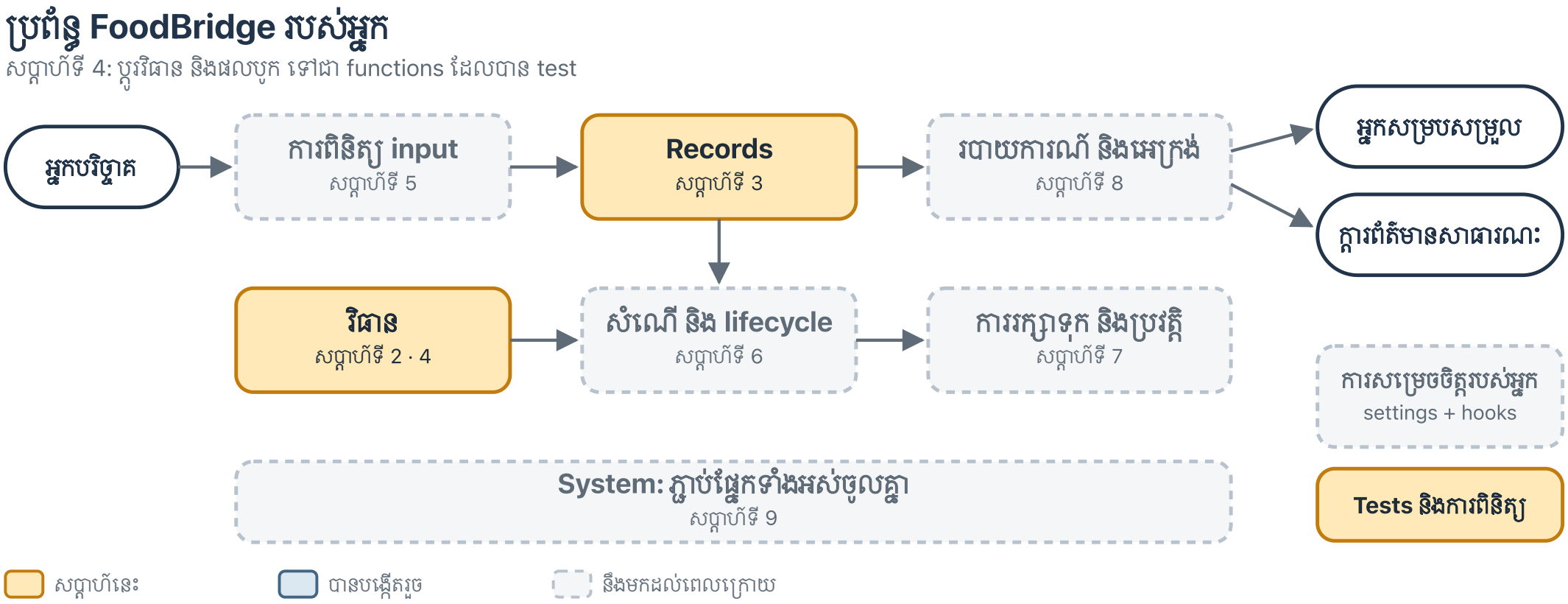
Programming for Information Systems

ការសន្យាច្បាស់លាស់ ចម្លើយតែមួយដែលត្រូវបាន return និង test ដែលបញ្ជាក់វា

សប្តាហ៍នេះ

សំណួរ	តើអ្នកអាចបង្ហាញដោយរបៀបណា ថា function មួយរក្សាការសន្យារបស់វា?
Python	`def`, parameter, `return` ធៀបនឹង `print`, `None`, docstring, `assert`, test function
គោលការណ៍ទី 4	function មួយ ការងារមួយ: return value, test ករណីធម្មតា boundary និងចម្លែក
អ្នកបង្កើត	`open_portions` និង test ផ្ទាល់ខ្លួនរបស់អ្នក
វគ្គទី 3	midterm ដែលធ្វើម្ចាក់ឯង

ប្រព័ន្ធរបស់អ្នក ត្រឹមសប្តាហ៍ទី 10



គំនិតសំខាន់: module និង contract

ផ្នាំងសាកទ្វរស័ព្ទសន្យាថា: ភ្លើងពីព្រីចូល ភ្លើងមានសុវត្ថិភាពចេញ **មិនដែល** ធ្វើឱ្យទ្វរស័ព្ទខូច។

contract (កិច្ចសន្យា) របស់ function និយាយថា:

- **Input:** អ្វីដែលអ្នកត្រូវតែផ្តល់ឱ្យវា។
- **Output:** អ្វីដែលវា return (និងពេលណាវា return `None`)។
- **មិនដែល:** អ្វីដែលវាមិនត្រូវធ្វើ ឧ. ប្តូរ list ដែលអ្នកផ្តល់ឱ្យ ឬ print។

> **module** គឺជា file មួយ ដែលមាន function ដែលមាន contract។ ផ្នែកដែលរក្សា contract អាចប្តូរជំនួសគ្នាបានដោយសុវត្ថិភាព។ **test** បង្ហាញថា contract ត្រូវបានរក្សា។

មូលដ្ឋាន Python

Function ដែលអាចទុកចិត្តបាន

Parameter និង argument

```python
def price_after_discount(price_riel, discount_percent):
    return price_riel - price_riel * discount_percent // 100

print(price_after_discount(10000, 10))     # 9000
print(price_after_discount(discount_percent=25, price_riel=4000))
print(price_after_discount(10, 10000))     # 🤔 ?
```

- **Parameter**: ឈ្មោះនៅក្នុងបន្ទាត់ `def` ។ **Argument**: value ក្នុងការហៅ។
- ផ្គូផ្គងតាម **ទីតាំង** លុះត្រាតែអ្នកផ្តល់វាតាមឈ្មោះ។

`return` ឬ `print`?

```python
def total_printed(prices_riel):
    total = 0
    for price_riel in prices_riel:
        total += price_riel
    print(total)

printed_result = total_printed([4000, 1500])   # បង្ហាញ 5500
print(printed_result)                          # None
```

- `print` **បង្ហាញ** value ដល់មនុស្ស រួចវាបាត់ទៅ។
- `return` **ផ្តល់** value ត្រឡប់ទៅវិញ ដូច្នេះ code អាចប្រើ ប្រៀបធៀប និង test វាបាន។

`None` មានន័យថា "គ្មានចម្លើយ"

```
prices_by_item = {"notebook": 4000, "pen": 1500}

def find_price(item_name):
    return prices_by_item.get(item_name)

price = find_price("ruler")
if price is None:
    print("Not sold here")
```

- `None` **ដោយចេតនា** គឺជាផ្នែកមួយនៃ contract: សរសេរវាទុក។
- `None` **ដោយចៃដន្យ** (ភ្លេច `return`) គឺជា bug។

Docstring: សរសេរការសន្យាទុក

```python
def average_mark(marks):
    """Return the average of a list of marks.

    Returns 0 for an empty list. Does not change the list.
    """
    if len(marks) == 0:
        return 0
    ...

help(average_mark)
```

- text ដំបូងនៅក្នុង function ក្នុងសញ្ញាសម្រង់បីជាន់។
- ប្រាប់ពី **output** ពី **ករណីចម្លែក** និងអ្វីដែលវា **មិនដែល** ធ្វើ។

Function ដែលហៅ function

```
URGENT_HOURS = 3

def is_urgent(hours_left):
    return hours_left <= URGENT_HOURS

def urgent_portions(records):
    total = 0
    for record in records:
        if is_urgent(record["hours_left"]):
            total += record["portions"]
    return total
```

- function នីមួយៗធ្វើ **ការងារមួយ** ហើយអាច test ដាច់ដោយឡែកបាន។

`assert`: ការពិនិត្យដែលឈប់ ហើយស្រែកប្រាប់

```python
assert price_after_discount(10000, 10) == 9000
assert average_mark([]) == 0
print("Both asserts passed")
# assert average_mark([50, 70]) == 50   # AssertionError
```

- `True` → គ្មានអ្វីកើតឡើង។ `False` → Python ឈប់ដោយ `AssertionError`។
- ការសន្យាដែលមិនត្រូវបានរក្សា មិនគួររំលងទៅដោយស្ងាត់ឈឹងទេ។

Test function: ធម្មតា boundary ទទេ

```
def test_average_normal():
    assert average_mark([60, 80]) == 70

def test_average_boundary():
    assert average_mark([0, 100]) == 50

def test_average_empty():
    assert average_mark([]) == 0

run_tests([test_average_normal, test_average_boundary, test_average_empty])
```

- ឈ្មោះចាប់ផ្តើមដោយ `test_`; ករណីមួយ ក្នុង test មួយ; ប្រើ `assert`។
- `run_tests` (ផ្តល់ឱ្យរួច) print ✅ ឬ ❌ សម្រាប់ test នីមួយៗ។ ផ្តល់ function ឱ្យវា **ដោយគ្មាន** `( )`។

បង្គត់ឡើង ហើយ test ដោយប្រើ constant

```
SEATS_PER_MINIBUS = 12

def minibuses_needed(students):
    return students // SEATS_PER_MINIBUS + 1   # ផ្លូវកាត់ដែលគួរឱ្យសាកល្បង

print(minibuses_needed(25))   # 3  ✓
print(minibuses_needed(24))   # 3  ✗ ទ្បានពេញពីរគត់
print(minibuses_needed(0))    # 1  ✗
```

- បង្គត់ឡើងដោយ `//` (ទ្បានពេញ) **និង** `%` (មាននរណានៅសល់ទេ?) ពីសប្តាហ៍ទី 1។
- test ដោយប្រើ constant: `minibuses_needed(SEATS_PER_MINIBUS + 1) == 2` មិនមែន `(13) == 2` ទេ។ ប្តូរវាទៅ 15: លេខ `13` ផ្ទាល់ នឹងខូច។ វាគឺជា **magic number**។

⚖️ ល្អ ឬមានហានិភ័យ?

Version A (ពីជំនួយការ AI)

```
def delivered_portions_a(records):
    total = 0
    for record in records:
        if record["status"] == "delivered":
            total += record["portions"]
            records.remove(record)
    print("Delivered:", total, "portions")

def test_delivered_a():
    delivered_portions_a(bridge.sample_records())
```

Version B

```
def delivered_portions_b(records):
    """Return delivered portions.
    Does not change records."""
    total = 0
    for record in records:
        if record["status"] == "delivered":
            total += record["portions"]
    return total

# test ប្លន: ធម្មតា, ពីរជាប់គ្នា,
# ទទេ, រក្សា list ដដែល
```

គោលការណ៍ទី 4 · function មួយ ការងារមួយ

- **ការងារមួយ:** គណនា ឬបង្ហាញ ឬប្តូរ។ មិនមែនទាំងបីទេ។
- **Return** value; ទុកឱ្យអ្នកហៅ សម្រេចថាត្រូវ print វាឬអត់។
- **កុំប្តូរ input** លុះត្រាតែនោះជាការងារតែមួយរបស់ function។
- **Test** ករណីធម្មតា boundary និងចម្លែក (ទទេ)។

> ពេលអ្នកពិនិត្យ code របស់ AI ចូរសួរ: *តើ test នេះពិនិត្យអ្វីពិតប្រាកដ?*

អ្នកបង្កើត: `open_portions` និង test ប្លន

```
OPEN_STATUSES = ["new", "approved", "picked_up"]

def open_portions(records):
    """Return the total portions of records whose status is in OPEN_STATUSES.

    Returns 0 for an empty list. Does not change records and does not print.
    """
    return None   # TODO

bridge.check("open_portions", open_portions)  # គំរូ → 42, [] → 0
```

- test របស់អ្នក: **ធម្មតា** (42), **boundary** (picked up រាប់ delivered មិនរាប់), **ទទេ** (0), **រក្សា list ដដែល**។

ភ្ជាប់ចូល: តើវាត្រូវគ្នា ពេញមួយថ្ងៃទេ?

```
compare("End of the sample day")      # 42
my_system.add_offer({... "portions": "16" ...})
compare("After a new offer arrives")  # 58
my_system.move("F001", "delivered")
compare("After F001 is delivered")    # 28
```

- `my_system.numbers()["portions_open"]` គឺជាលេខផ្ទាល់ខ្លួនរបស់ប្រព័ន្ធ។
- function ដែលអ្នកអាចទុកចិត្ត ត្រូវគ្នា **ក្រោយគ្រប់ការប្រែប្រួល** មិនមែនតែលើទិន្នន័យគំរូទេ។

🧭 ការពិភាក្សា · តើ status ណាខ្លះ រាប់ថា open?

សប្តាហ៍នេះ គ្មាន setting ត្រូវសម្រេចទេ: "open" ស្ថិតនៅក្នុង `OPEN_STATUSES` ។ ចូររាយតម្លៃវា។

1. តើ **picked up** គួររាប់ទេ? អាហារនៅក្នុងកន្ត្រក មិនទាន់ដល់អង្គការដែគួរទេ។
2. តើ **new** គួររាប់ទេ? អ្នកសម្របសម្រួលនៅតែអាចបដិសេធវាបាន។
3. តើអ្នកណាអានលេខនេះ ហើយពួកគេធ្វើអ្វីជាមួយវា?

> រក្សានិយមន័យរបស់ប្រព័ន្ធ សម្រាប់ check។ ក្នុងកំណត់ចំណាំរបស់អ្នក: តើអ្នកយល់ស្របទេ ជម្រើសផ្សេងគឺអ្វី ហើយវានឹងជួយ ឬប៉ះពាល់ដល់អ្នកណា?

វគ្គទី 3 · Midterm · រៀបចំផែនការសម្រាប់មួយថ្ងៃ

ផ្នែក	អ្វីដែលអ្នកធ្វើ	ពិន្ទុ
A · អាន code	ពន្យល់ថា code ខ្លីៗប្រាំ print អ្វី ហើយ **ហេតុអ្វី**	20
B · សរសេរវិធាន	function តូចមួយ ដែលមាន constant មានឈ្មោះ	20
C · ធ្វើការជាមួយ record	សង្ខេប list នៃសំណើ ដោយមិនប្តូរវា	25
D · test ដែលចាប់ bug	test ឬន run លើ bug ដែលដាក់លាក់ទុក	15
E · វាយតម្លៃ code របស់ AI	រកបញ្ហាក្នុង code ដែល run បាន	10
F · ការសម្រេចចិត្តរបស់អ្នក	ជ្រើស constant មួយ, restart session and run all, កំណត់ត្រាការសម្រេចចិត្តខ្លី	10

- **ម្នាក់ឯង។** អនុញ្ញាត: កំណត់ចំណាំរបស់អ្នក notebook ប្រចាំសប្តាហ៍ Python Basics។ មិនអនុញ្ញាត: ឧបករណ៍ AI ការផ្ញើសារ។
- បន្ទាប់មក **ការពិនិត្យថ្នាល់មាត់ 3 នាទី**: ពន្យល់បន្ទាត់មួយនៃ code របស់អ្នក និង test មួយ។

កិច្ចការសប្តាហ៍នេះ

- ⭐ **មូលដ្ឋាន:** P1–P4, បញ្ហាចីក្នុង Version A, `open_portions` ជាប់ ✅, test ប្លងដែលជាប់, ត្រូវនឹងប្រព័ន្ធ, កំណត់ចំណាំការពិភាក្សាអំពី "open"។
- ⭐⭐ **បន្ថែម:** S1 (test ដែលចាប់ bug); `portions_with_status` ដោយ test របស់អ្នកនៅតែជាប់។
- 📝 **វគ្គទី 3:** midterm ក្នុង `Midterm.ipynb`។
- 🔎 **កិច្ចការស្រាវជ្រាវទី 3 (5%) ដាក់ឱ្យធ្វើក្រោយ midterm:** ការពិនិត្យ input លើ form ពិតមួយ។ ចែករំលែករវាងពេលចាប់ផ្តើមថ្នាក់សប្តាហ៍ទី 5។

សប្តាហ៍ក្រោយ · ការពិនិត្យ input

input ពិតប្រាកដ គឺរញ៉េរញ៉ៃ: ដកឃ្លាលើស ពាក្យជំនួសឱ្យលេខ field ដែលបាត់។ អ្នកនឹងពិនិត្យ **គ្រប់សំណើនៅមាត់ទ្វារ** ហើយមិនប្តូរអ្វីទាំងអស់ ពេល input មិនត្រឹមត្រូវ។